

p38.js

```
const fruits=["사과", "오렌지", "배", "딸기", "수박"];
// 1. 배열 자르기
const selected =fruits.slice(1, 4);
console.log(selected);
// ["오렌지", "배", "딸기"] => 1, 2, 3
console.log(fruits);
// ["사과", "오렌지", "배", "딸기", "수박"]
// 2. 배열 합치기
const tropical=["망고", "바나나"];
const allFruits =fruits.concat(tropical);
console.log(allFruits);
// ["사과", "오렌지", "배", "딸기", "수박", "망고", "바나나"]
```

p38.ts

```
const fruits:string[]=["사과", "오렌지", "배", "딸기", "수박"];
// 1. 배열 자르기
const selected:string[] =fruits.slice(1, 4);
console.log(selected);
// ["오렌지", "배", "딸기"]
console.log(fruits);
// ["사과", "오렌지", "배", "딸기", "수박"]
// 2. 배열 합치기
const tropical:string[]=["망고", "바나나"];
const allFruits:string[] =fruits.concat(tropical);
console.log(allFruits);
// ["사과", "오렌지", "배", "딸기", "수박", "망고", "바나나"]
```

p38 result

```
[ '오렌지', '배', '딸기' ]
[ '사과', '오렌지', '배', '딸기', '수박' ]
[
  '사과',   '오렌지',
  '배',     '딸기',
  '수박',   '망고',
  '바나나'
]
```

p40.js

```
const text ="JavaScript";
```

```
// 문자열 길이
console.log(text.length); // 10
// 인덱스로 문자 찾기
console.log(text[0]); // J
console.log(text[3]); // a
// 마지막 문자 찾기
console.log(text[text.length - 1]); // t
console.log(text.at(-1)); // t
// 문자 또는 문자열 찾기
console.log(text.includes("Script")); // true
console.log(text.indexOf("a")); // 1
console.log(text.lastIndexOf("a")); // 3
```

p40.ts

```
const text:string ="JavaScript";
// 문자열 길이
console.log(text.length); // 10
// 인덱스로 문자 찾기
console.log(text[0]); // J
console.log(text[3]); // a
// 마지막 문자 찾기
console.log(text[text.length - 1]); // t
console.log(text.at(-1)); // t
// 문자 또는 문자열 찾기
console.log(text.includes("Script")); // true 문자가 포함
console.log(text.indexOf("a")); // 1 인덱스 위치
console.log(text.lastIndexOf("a")); // 3 마지막 인덱스 위치
```

p40 result

```
10
J
a
t
t
true
1
3
```

p42.js

```
const fruits=["사과", "오렌지", "배", "딸기"];
// 1. 배열 → 문자열
```

```
const text1 =fruits.join();
const text2 =fruits.join(" / ");
console.log(text1);
// 사과,오렌지,배,딸기
console.log(text2);
// 사과 / 오렌지 / 배 / 딸기
// 2. 문자열 → 배열
const fruits2 =text2.split(" / ");
console.log(fruits2);
// ["사과", "오렌지", "배", "딸기"]
```

p42.ts

```
const fruits:string[] =["사과", "오렌지", "배", "딸기"];
// 1. 배열 → 문자열
const text1:string =fruits.join();
const text2:string =fruits.join(" / ");
console.log(text1);
// 사과,오렌지,배,딸기
console.log(text2);
// 사과 / 오렌지 / 배 / 딸기
// 2. 문자열 → 배열
const fruits2:string[] =text2.split(" / ");
console.log(fruits2);
// ["사과", "오렌지", "배", "딸기"]
```

p42 result

사과,오렌지,배,딸기

사과 / 오렌지 / 배 / 딸기

['사과', '오렌지', '배', '딸기']

p44.js

```
const scores =[[80, 90], [100, 80], [90, 70]];
// 1. 중첩 배열을 1차원 배열로 변환
const flatScores =scores.flat();
console.log(flatScores);
// [80, 90, 100, 80, 90, 70]
// 2. 배열을 특정 값으로 채우기
const result =new Array(5).fill(0);
console.log(result);
// [0, 0, 0, 0, 0]
```

```
// 3. Set으로 중복 제거
const uniqueScores = new Set(flatScores);
console.log(uniqueScores);
// Set {80, 90, 100, 70}
console.log(uniqueScores.size);
// 4
// 4. Set을 다시 배열로 변환
const finalScores = Array.from(uniqueScores);
console.log(finalScores);
// [80, 90, 100, 70]
```

p44.ts

```
const scores:number[][] =[
[80, 90],
[100, 80],
[90, 70]
];
// 1. 중첩 배열을 1차원 배열로 변환
const flatScores:number[] =scores.flat();
console.log(flatScores);
// 2. 배열을 특정 값으로 채우기
const result:number[] =new Array(5).fill(0);
console.log(result);
// 3. Set으로 중복 제거
const uniqueScores:Set<number> =new Set(flatScores);
console.log(uniqueScores);
console.log(uniqueScores.size);
// 4. Set을 다시 배열로 변환
const finalScores:number[] =Array.from(uniqueScores);
console.log(finalScores);
```

p44 result

```
[ 80, 90, 100, 80, 90, 70 ]
[ 0, 0, 0, 0, 0 ]
Set(4) { 80, 90, 100, 70 }
4
[ 80, 90, 100, 70 ]
```

p46.ts

```
// forEach 함수에서 첫 번째는 요소 두 번째는 인덱스 번호 세 번째는 배열 전체
출력
```

```
function printNames(namesArr:string[]):void {
  namesArr.forEach( (name, index, arr) =>console.log(name + " : "+index
  +"=> "+arr),
  );
}
printNames(["kim", "lee", "park"]);
// 익명 함수로 콜백함수 생성
const arr=["가", "나", "다"];
arr.forEach(function(element) {
  console.log(element);
});
```

p46 result

```
kim : 0=> kim,lee,park
lee : 1=> kim,lee,park
park : 2=> kim,lee,park
가
나
다
```

p47.ts

```
const numbers:number[] =[1, 2, 3, 4];
const result =numbers.map( (num) =>{
  return num *2;
});
console.log(result);
// map 줄여서 표현
const result_short =numbers.map((num) =>num *2);
console.log(result_short);
// Return 이 없으면 result2 는 빈값이 들어감
const result2 =numbers.map( (num) =>{
  num *2;
});
console.log(result2);
// map 연습
const result3 =numbers.map((e, i) =>{
  return `${e}의 세 배 제공은 ${e** 3}${i}번째 인덱스`;
})
console.log(result3)
```

p47 result

[2, 4, 6, 8]

[2, 4, 6, 8]

[undefined, undefined, undefined, undefined]

[

 '1의 세 제공은 1 0번째 인덱스',

 '2의 세 제공은 8 1번째 인덱스',

 '3의 세 제공은 27 2번째 인덱스',

 '4의 세 제공은 64 3번째 인덱스'

]

p48.ts

```
const scores:number[] =[65, 80, 95, 70, 90];
// find()
const firstHighScore =scores.find(score =>score >=90);
console.log(firstHighScore);
// findIndex()
const highScoreIndex =scores.findIndex(score =>score >=90);
console.log(highScoreIndex);
// filter()
const highScores:number[] =scores.filter(score =>score >=90);
console.log(highScores);
```

p48 result

95

2

[95, 90]

p49.ts

```
const scores:number[] =[80, 100, 70, 90, 60];
scores.sort((a, b) =>b -a); // b - a 내림차순
console.log(scores);
scores.sort((a, b) =>a -b); // a - b 오름차순
console.log(scores);
// 비교 함수 사용 안함
const scores2:number[] =[80, 100, 70, 90, 60];
scores2.sort(); // 기본 문자열 비교 방식으로 정렬될 수 있음
console.log(scores2); // [ 100, 60, 70, 80, 90 ]
const scores1:number[] =[80, 100, 70, 90, 60];
const total:number =scores1.reduce(
  (sum, score) =>sum +score,
  0
```

```
);  
console.log(total);
```

p49 result

[100, 90, 80, 70, 60]

[60, 70, 80, 90, 100]

[100, 60, 70, 80, 90]

400

p50.ts

```
const scores:number[] =[80, 90, 70, 100];  
const result:boolean =scores.every(  
score =>score >=60  
);  
console.log(result);  
  
const scores1:number[] =[50, 60, 70, 95];  
const result1:boolean =scores.some(  
score =>score >=90  
);  
console.log(result);
```

p50 result

true

true

p52.ts

```
let tuple:[string, number, boolean] =["jamsuham", 40, true];  
console.log(tuple[0]);  
console.log(tuple[1]);  
console.log(tuple[2]);  
let optionalTuple:[string, number?] =["jamsuham"];  
console.log(optionalTuple);  
optionalTuple =["aaa"];  
console.log(optionalTuple);  
optionalTuple =["bbb", 40];  
console.log(optionalTuple);
```

p52 result

jamsuham

40

true

```
[ 'jamsuham' ]  
[ 'aaa' ]  
[ 'bbb', 40 ]
```

p53.ts

```
const student:[string, number] =["홍길동", 20];  
const[studentName, studentAge] =student;  
console.log(studentName); // 홍길동  
console.log(studentAge); // 20  
function getUser():[string, number] {  
    return["홍길동", 20];  
}  
const[userAge, userAge] =getUser();  
console.log(userAge); // 홍길동  
console.log(userAge); // 20  
// 연습해보기  
function getUser2(name :string, num :number) :[string, number] {  
    return[name, num];  
}  
const[userAge2, userAge2] =getUser2("박수진", 25);  
console.log(userAge2);  
console.log(userAge2);
```

p53 result

```
홍길동  
20  
홍길동  
20  
박수진  
25
```

p54.ts

```
// 튜플에 forEach  
const student:[string, number] =["홍길동", 20];  
student.forEach((value) =>{  
    console.log(value);  
})  
// 튜플에 map  
const numbers:[number, number, number] =[10, 20, 30];  
const result =numbers.map(num =>num *2);  
console.log(result);  
// 연습해보기
```

```
const products :[string, string, string] =["샴푸", "폼클렌징", "바디워시"];
products.forEach((e) =>{
    console.log(e);
})
const lengthPlus =products.map(p =>`${p}는 ${p.length}개 챙겨`);
console.log(lengthPlus);
```

p54 result

홍길동

20

[20, 40, 60]

샴푸

폼클렌징

바디워시

['샴푸는 2개 챙겨', '폼클렌징은 4개 챙겨', '바디워시는 4개 챙겨']

p55.ts

// 튜플에 includes, indexOf 사용

```
const fruits:[string, string, string] =["사과", "오렌지", "딸기"];
```

```
console.log(fruits.includes("사과"));
```

```
console.log(fruits.indexOf("딸기"));
```

// 튜플에 슬라이싱

```
const student:[string, number, string] =["홍길동", 20, "서울"];
```

```
const result =student.slice(0, 2); // 0~1번째 인덱스
```

```
console.log(result);
```

// 연습해보기

```
const games :[string, string, string] =["League of Legends", "Maple Story", "Lost Ark"]
```

```
console.log(games.includes("MineCraft"));
```

```
console.log(games.indexOf("Lost Ark"));
```

```
console.log(games.slice(0,2));
```

p55 result

true

2

['홍길동', 20]

false

2

['League of Legends', 'Maple Story']

p5.ts

```
const book:// book 객체 생성
  title:string; // 제목 문자열
  author:string; // 저자 문자열
  year:number; // 출판년도 숫자
  publisher?:string; // 출판사(없어도 무방) 문자열
} = {
  title:"프론트엔드프로그래밍",
  author:"동미대",
  year:2026
};
// 1. 객체 속성 접근 - 마침표(.) 사용
console.log(book.title); // 프론트엔드프로그래밍
console.log(book.author); // 동미대
console.log(book.year); // 2026
// 대괄호([]) 사용
console.log(book["title"]); // 프론트엔드프로그래밍
console.log(book["author"]); // 동미대
// 2. 객체 속성 추가하기
book.publisher = "DMU출판사";
console.log(book.publisher); // DMU출판사
// 3. 객체 속성 수정하기
book.author = "홍길동";
book.year = 2027;
console.log(book.author); // 홍길동
console.log(book.year); // 2027
// 4. 객체 속성 삭제하기
delete book.publisher
;
console.log(book.publisher); // undefined
```

p5 result

프론트엔드프로그래밍

동미대

2026

프론트엔드프로그래밍

동미대

DMU출판사

홍길동

2027

undefined

p8.ts

```
const book:{
  title:string;
  author:string;
  year:number;
// 중첩 객체
  info:{
    category:string;
    price:number;
};
// 선택적 속성
  publisher?:{
    name:string;
    address?:string;
};
// 메서드
getBookInfo(): =>string;
} = {
  title:"프론트엔드프로그래밍",
  author:"동미대",
  year:2026,
  info:{
    category:"프로그래밍",
    price:30000
  },
  getBookInfo:function() {
    return `${this.title}${this.author}`;
  }
};
// 1. 중첩 객체의 속성 접근
console.log(book.info.category); // 프로그래밍
console.log(book["info"]["price"]); // 30000
// 마침표와 대괄호 조합
console.log(book.info["category"]); // 프로그래밍
console.log(book["info"].price); // 30000
// 2. ?. (옵셔널 체이닝) : 자바스크립트, 타입스크립트 모두 사용
```

```

console.log(book.publisher?.name); // undefined
console.log(book.publisher?.address); // undefined
// 3. 객체의 메서드 호출
console.log(book.getBookInfo());
// 프론트엔드프로그래밍 / 동미대
// 4. 객체간 비교
const book2 = {
  title:"프론트엔드프로그래밍",
  author:"동미대",
  year:2026
};
const book3 =book;
console.log(book ===book2); // false
console.log(book ===book3); // true

```

p8 result
 프로그래밍
 30000
 프로그래밍
 30000
 undefined
 undefined
 프론트엔드프로그래밍 / 동미대
 false
 true

p9.js

```

const book = {
  title:"프론트엔드프로그래밍",
  author:"동미대",
  year:2026,
  info:{
    category:"프로그래밍",
    price:30000
  },
  getBookInfo:function() {
    return `${this.title}${this.author}`;
  }
};
// 1. 중첩 객체의 속성 접근

```

```

console.log(book.info.category); // 프로그래밍
console.log(book["info"]["price"]); // 30000
// 마침표와 대괄호 조합
console.log(book.info["category"]); // 프로그래밍
console.log(book["info"].price); // 30000
// 2. ?. (옵셔널 체이닝) : 자바스크립트, 타입스크립트 모두 사용
console.log(book.publisher?.name); // undefined
console.log(book.publisher?.address); // undefined
// 3. 객체의 메서드 호출
console.log(book.getBookInfo()); // 프론트엔드프로그래밍 / 동미대
// 4. 객체간 비교
const book2 = {
  title: "프론트엔드프로그래밍",
  author: "동미대",
  year: 2026
};
const book3 = book;
console.log(book === book2); // false
console.log(book === book3); // true

```

p9 result
 프로그래밍
 30000
 프로그래밍
 30000
 undefined
 undefined
 프론트엔드프로그래밍 / 동미대
 false
 true

p10.ts
 // 객체로 지정하면 주소값이 같음 변경해도 따라감
 const a = {name: "홍길동"};
 const b = a;
 console.log(a.name);
 console.log(b.name);
 a.name = "김철수"
 console.log(a.name);
 console.log(b.name);

```
console.log(a===b); //true
```

p10 result

홍길동

홍길동

김철수

김철수

true

p11.ts

// 객체가 아닌 값을 대입할 경우 변경해도 따라가지 않음

```
let a:string = "홍길동";
```

```
let b:string = a;
```

```
console.log(a); // 홍길동
```

```
console.log(b); // 홍길동
```

```
a = "김철수";
```

```
console.log(a); // 김철수
```

```
console.log(b); // 홍길동
```

p11 result

홍길동

홍길동

김철수

홍길동

p12.ts

```
const students = // 배열에 객체 생성
```

```
{ name:"홍길동"},
```

```
{ name:"김철수"}
```

```
];
```

```
const reference = students;
```

```
console.log(students === reference); // true
```

```
reference[0].name = "이영희";
```

```
console.log(students[0].name); // 이영희
```

p12 result

true

이영희

p13.ts

```
const user:{
```

```
name:string;
```

```
age:number;
```

```
} = {
```

```

name:"홍길동",
age:20
};
// 기존 방식
const userName =user.name;
const userAge =user.age;
console.log(userName); // 홍길동
console.log(userAge); // 20
// 객체 구조 분해 할당
const {name: nName, age: nAge}=user;
console.log(nName); // 홍길동
console.log(nAge); // 20
const fruits=["사과", "오렌지", "딸기"]; // 배열
const data:// 객체
0:string;
1:string;
2:string;
length:number;
} ={}
0:"사과",
1:"오렌지",
2:"딸기",
length:3,
};
console.log(data[0]); // 사과
console.log(data.length); // 3
console.log(Array.isArray(fruits)); // true
console.log(Array.isArray(data)); // false

```

p13 result

홍길동
 20
 홍길동
 20
 사과
 3
 true
 false